

Requirements, STLC & Software Environments

From Requirement to Production Confidence

PM Session · 13:30–17:00

- Requirement artifacts & levels
- Software Testing Life Cycle
- Dev, Test & Production environments

Running example throughout

 Money Transfer System (MTS) — a teaching vehicle, not the subject

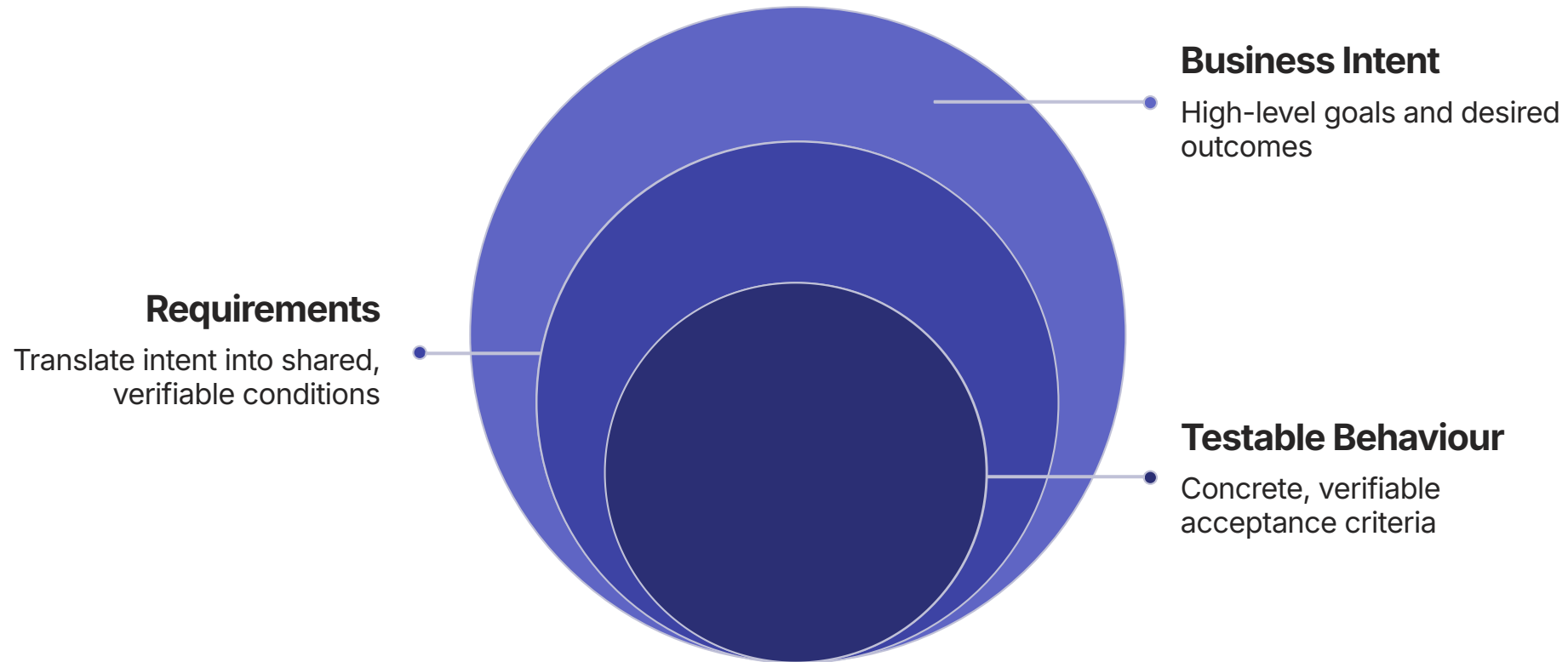
CONTINUING FROM DECK 1

Where does a tester learn what the system *should* do?

Deck 1 ended here: **Business Need** → **Requirements** → **Build** → **Test** → **Production**

📄 Deck 2 answers this question fully — from raw business need to production confidence.

Requirements are the source of expected behaviour



Requirements translate intent into shared, verifiable understanding — but they exist at different levels and in different forms.

MTS EXAMPLE

A business stakeholder says...

"We need customers to be able to transfer money."

Can QA test this sentence completely? 🤔

What's missing from that one sentence?

Who?

Which users can transfer? Authenticated only? Account type?

How much?

Minimum? Maximum? Daily limits? Per transaction?

What if it fails?

Insufficient balance? Invalid receiver? Network error?

Quality bar?

How fast? How secure? How available?

A single sentence is business *intent* — not a testable specification.

Requirements turn intent into shared, testable understanding

Shared

Business, Dev, and QA all working from the same truth

Precise

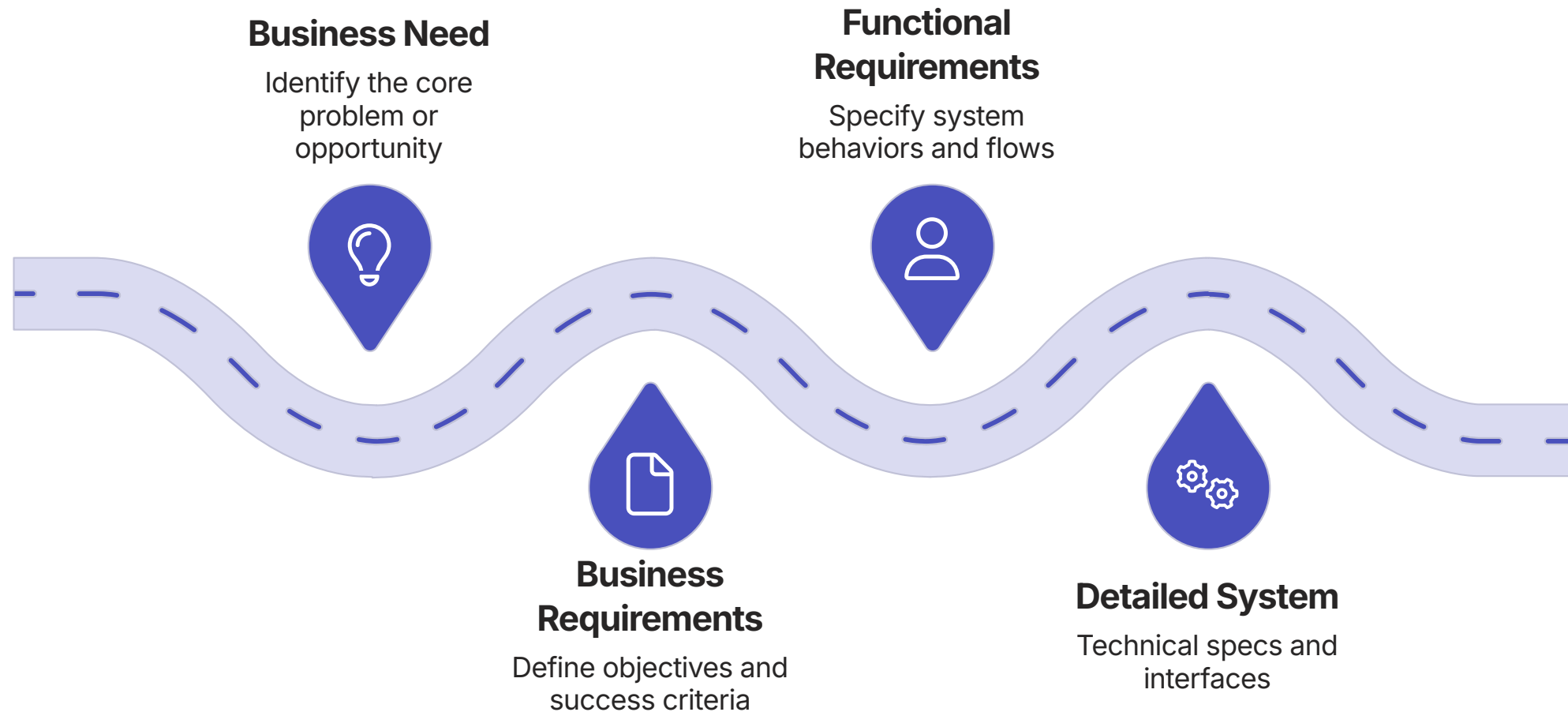
Unambiguous enough to build and to test

Testable

Each requirement can be verified — pass or fail

- ❏ Without clear requirements, testers guess. Guessing = missed defects and wasted effort.

Requirements Flow — From Need to Evidence



Every test case traces back to a real business need. Every business need eventually becomes testable evidence.

Requirements exist at different levels

1

System / Detail Level

Specific behaviours, constraints, interfaces

2

Functional / User Level

What the system does for users

3

Business Level

WHY — goals, outcomes, scope

 Document names vary widely by organization — BRD, FRD, SRS are common labels but not universal standards.

PROBLEM FIRST

Business stakeholders need to agree on *WHY* and *what outcome* is needed

Before anyone builds or tests anything, the business case must be documented and agreed upon.

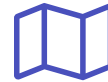
Enter the **Business Requirements Document**.

BRD — Business Requirements Document



Business Objectives

The WHY — what problem are we solving?



Scope & Stakeholders

What is in/out? Who is involved?



High-Level Capabilities

What the system must enable at business level



Constraints & Success Criteria

Boundaries and how we know we succeeded



Templates and depth vary. Some orgs call this a "Business Case" or fold it into a product charter.

MTS EXAMPLE — BRD LEVEL

MTS Business Requirements (BRD style)

Objective

Enable customers to initiate secure domestic money transfers via digital channels

Business Goal

Reduce manual processing time and branch dependency

Scope

Domestic transfers only; excludes international, forex, and batch payroll

Notice: *no* validation rules, no field specs, no API details — that comes next.

BRD tells us WHAT the business needs.

How will the *application* behave? 🤔

FRD — Functional Requirements Document

Describes **expected system functions and behaviours**, including business rules that govern them.

- What the system does in response to inputs
- Business rules applied during processing
- How the system interacts with users/other systems
- Expected outcomes for each behaviour

BRD → FRD shift

BRD: "Enable secure transfers"

FRD: "**System shall validate sender account is active and has sufficient balance before processing**"

MTS Functional Requirements

1 Validate accounts

Sender account must be active; receiver account must exist and be eligible

2 Validate amount

Amount must be positive, within limits, and not exceed available balance

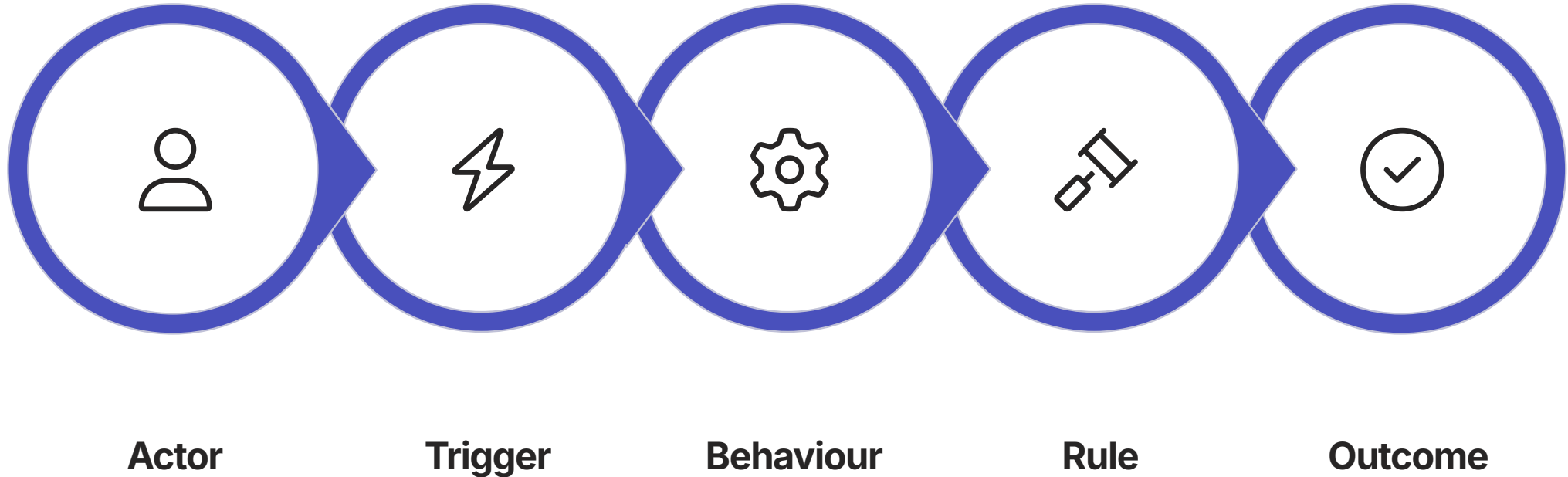
3 Process atomically

Debit sender and credit receiver must succeed together or both roll back

4 Return outcome

System returns success confirmation or a specific, actionable error code

Anatomy of a good functional requirement



Example: "When an authenticated customer (actor) submits a transfer request (trigger), the system shall debit the sender and credit the receiver (behaviour) only if the amount is within the daily limit (rule), and return a success reference number (outcome)."

Ambiguous vs. testable requirements

Ambiguous

"The transfer should be fast."

"The system should handle errors gracefully."

"Transfers must be secure."

Testable

"95% of transfers complete within 3 seconds under normal load."

"System returns error code E004 with a user-readable message on balance failure."

"All transfer data transmitted over TLS 1.2 or higher."

If you cannot write a pass/fail test for it — it is not testable enough.

Where do *detailed* system behaviours, interfaces, and constraints get captured?

SRS — Software Requirements Specification

Detailed Functional Requirements

Precise system behaviours, validations, edge cases

Non-Functional Requirements

Performance, security, scalability, availability, compliance

System Interfaces

External systems, APIs, data formats, integration points

Constraints & Assumptions

Platform, regulatory, or operating constraints

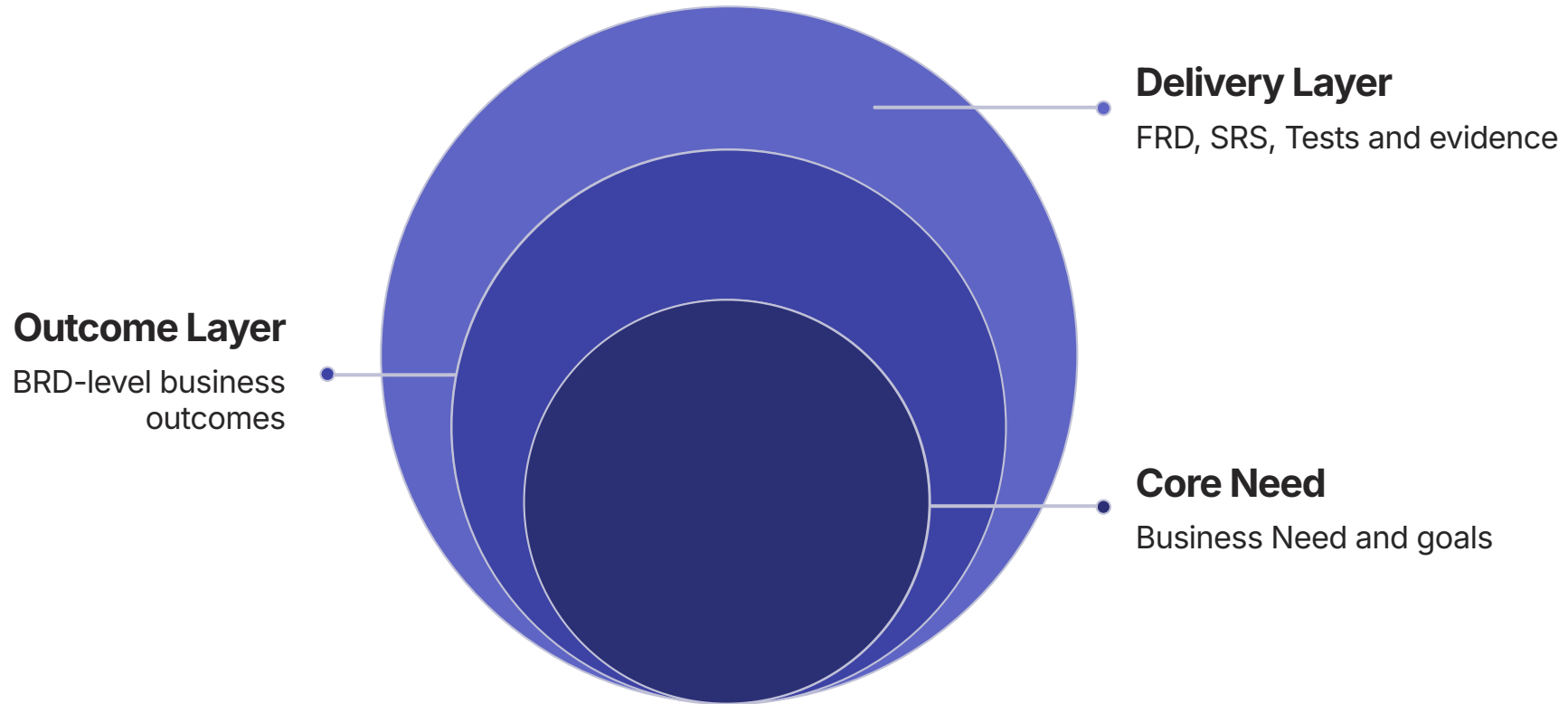
- ❏ SRS specifies WHAT the software must do — not HOW it is designed or built. Architecture documents cover HOW.

BRD vs FRD vs SRS — at a glance

Dimension	BRD	FRD	SRS
Core question	WHY?	WHAT does it do?	WHAT exactly, in detail?
Audience	Business stakeholders	BA, Dev, QA	Dev, QA, Architects
Detail level	High-level outcomes	Functional behaviours	Detailed specification
Includes NFRs?	Rarely	Sometimes	Yes

 Organizations combine, rename, or skip these. The concepts matter more than the document names.

Requirements build toward tests — not all at once



Key reminder: Not every project produces all three documents separately. Agile teams may combine or replace them entirely with user stories and acceptance criteria.

REQUIREMENT TYPE 1

Functional Requirements

What the system does — features, behaviours, responses, business rules

01

Input

User initiates a transfer of ₹5,000

02

System behaviour

Validates account, balance, and limits

03

Output

Transfer confirmed; reference number returned

Non-Functional Requirements — the quality bar



Performance

Response time, throughput, latency under load



Availability

Uptime, recovery time objectives, failover



Compliance

Regulatory, audit, data protection obligations



Security

Authentication, encryption, authorization rules



Scalability

Handles increased users/data without degradation



Usability

Learnability, accessibility, error recovery

Functional vs. Non-Functional — same feature, two lenses

Functional

"The system shall transfer ₹X from sender to receiver and return a transaction ID."

Tests: did it work? did it record correctly? did it reject invalid inputs?

Non-Functional

"95% of transfer requests shall complete within 3 seconds under 500 concurrent users."

Tests: does it meet the quality bar? at scale? under stress?

Both are requirements. Both need to be tested. Neither is optional.

Requirements can also appear as User Stories

In Agile teams, large requirement documents are often replaced by **lightweight user stories** that capture intent from the user's perspective.

Stories are short, collaborative, and refined through conversation — not exhaustive upfront documentation.

BRD / FRD / SRS

Heavyweight documents, upfront specification, commonly used in structured/regulated environments

User Stories + Acceptance Criteria

Lightweight, iterative, conversation-driven; common in Agile/Scrum teams

USER STORY STRUCTURE

**As a [user],
I want [capability],
so that [value].**

Simple. Focused on the user. Conversation-starter, not a complete specification.

User Story: Customer initiates a transfer

**As a registered customer,
I want to transfer money to another account,
so that I can pay people without visiting a branch.**

Clear intent. User-centred. But... is this enough to test from?

A user story alone leaves too many questions unanswered.

What counts as a successful transfer? When should it be rejected? What messages does the user see?

Acceptance Criteria fill that gap.

Acceptance Criteria — conditions for acceptance

Define the **minimum conditions** under which the story is considered done and working.

One useful format — **Given / When / Then**:

 Given/When/Then (Gherkin) is one popular format — not a mandatory universal standard. Some teams write acceptance criteria as simple bullet conditions.

Given

The precondition / starting state

When

The action the user/system takes

Then

The expected observable outcome

MTS Acceptance Criteria

1

Valid transfer succeeds

Given active sender with sufficient balance and valid receiver — transfer completes and a reference number is returned

2

Insufficient balance rejected

Given sender balance below the requested amount — transfer is declined with error code E001

3

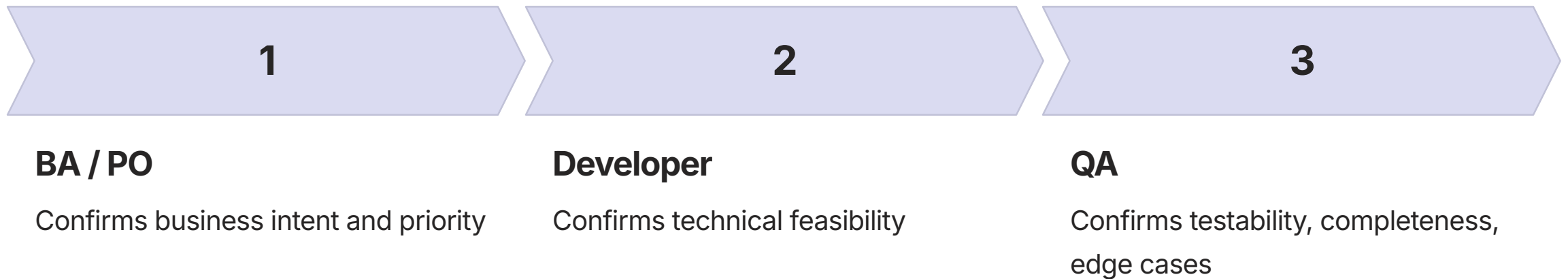
Invalid receiver rejected

Given a receiver account that does not exist — transfer is declined with error code E002

Each criterion maps directly to at least one test case.

What if requirements *contradict* each other, omit edge cases, or simply cannot be tested? 🤔

Requirement Review — before a single line of code is written



All relevant stakeholders align **before** implementation. Defects caught here cost almost nothing compared to defects found in production.

Six questions QA asks during requirement analysis



Clear?

Only one interpretation?



Complete?

All scenarios covered?



Consistent?

No contradictions?



Feasible?

Can it be built?



Testable?

Can we prove pass/fail?



Error paths?

What happens when things go wrong?

Requirement: "Allow transfer up to daily limit."

- What **is** the daily limit? Is it configurable per user tier?
- Which **timezone** defines the day? UTC? Local? Bank timezone?
- Is the limit per **user or per account**? Joint account?
- Is the boundary **inclusive**? (₹50,000 limit — does ₹50,000 transfer succeed?)
- When does the limit **reset**? Midnight? Rolling 24 hours?

📌 One ambiguous sentence. Five critical questions. This is exactly QA's role at requirement time.

Testing begins *before* test execution.

QA prevents defects by **questioning requirements early** — when the cost of change is lowest.

Found in Requirements

Low cost, quick fix
when caught early

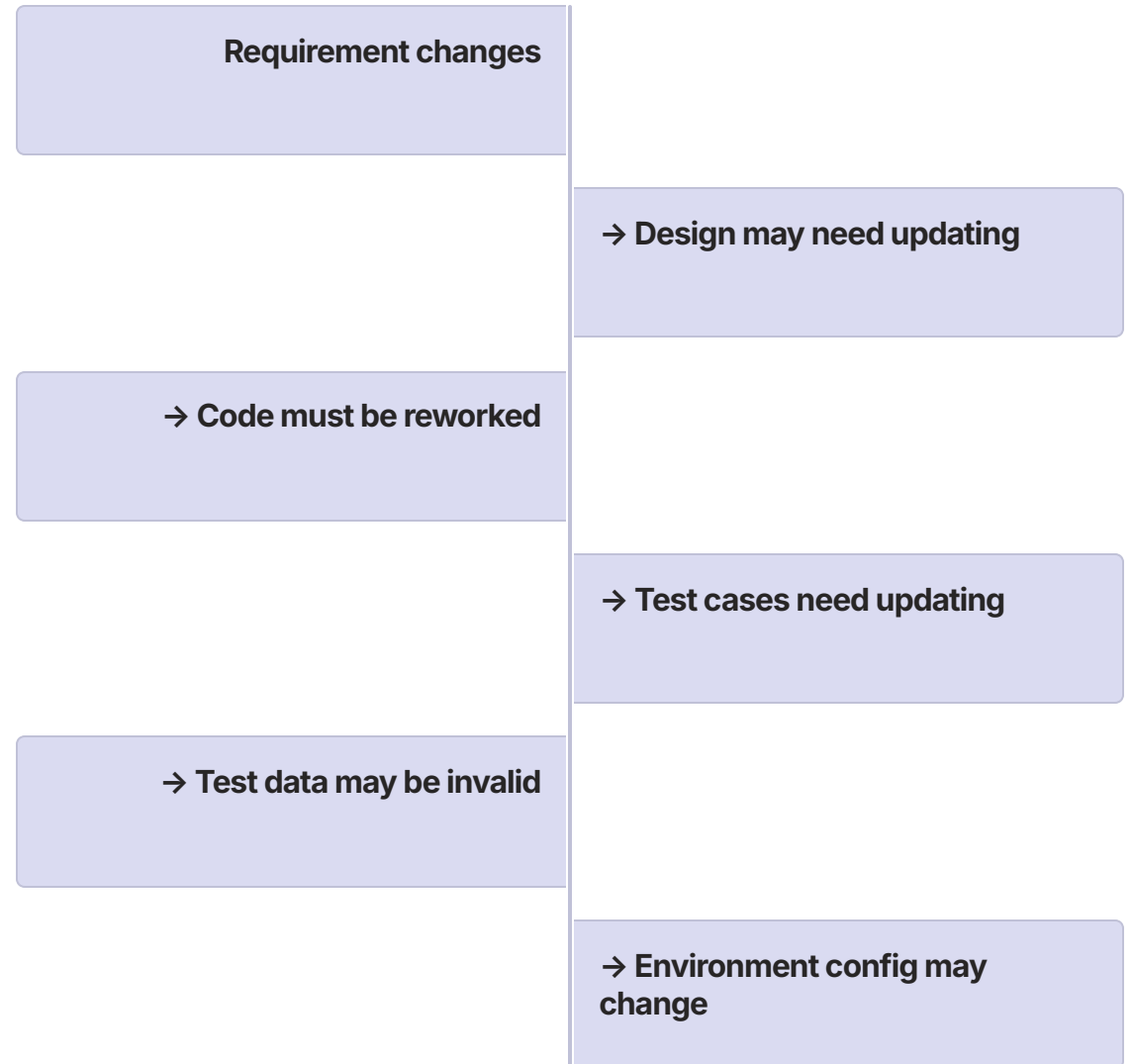


Found in Production

High cost,
customer impact
and remediation
delays

Requirements change — and change has ripple effects

A changed requirement doesn't just affect one thing. It can cascade across the entire delivery pipeline.



**With dozens or hundreds of
requirements...**

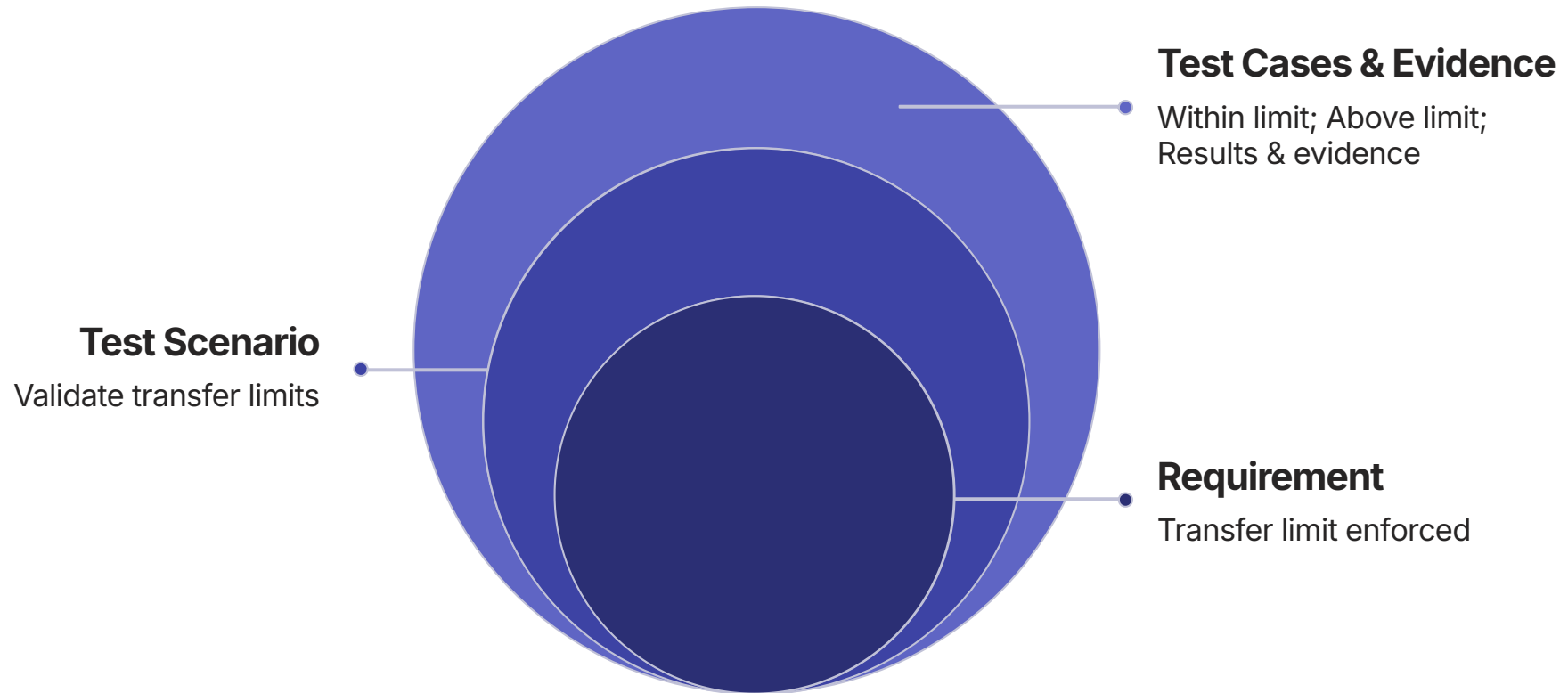
**How do we *prove* each
one was tested? 🤔**

CORE CONCEPT

Traceability

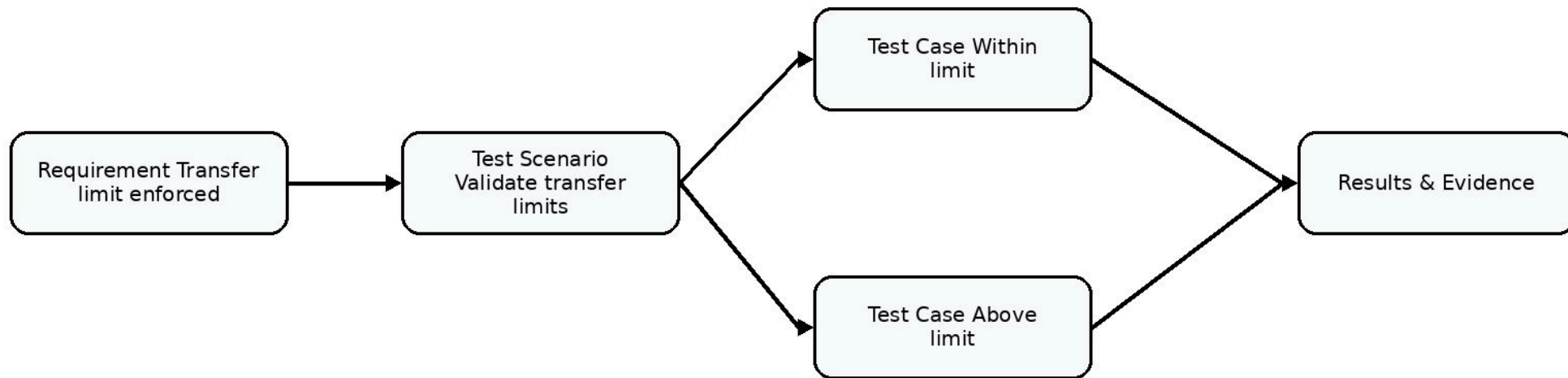
The discipline of linking every requirement to the tests that verify it — and every test back to the requirement that justifies it.

Requirement Traceability — MTS Example



Every result links back to a requirement. Every requirement links forward to evidence. This is the chain of proof.

Requirement Traceability — Follow the Requirement



RTM — Requirements Traceability Matrix

Requirement	Test Scenario	Test Cases	Result
Transfer limit enforced	Validate transfer limits	TC-01, TC-02	Pass 
Invalid receiver rejected	Receiver validation	TC-03	Pass 
Atomic debit/credit	Rollback on failure	TC-04, TC-05	In Progress 

📄 Format and tooling vary (spreadsheet, Jira, Xray, Zephyr) — the concept of linking requirements to evidence is universal.

Traceability answers two directions

↓ Forward Traceability

"Did we test **every requirement**?"

Requirement → Scenarios → Test Cases → Evidence

↑ Backward Traceability

"Why does **this test exist**?"

Evidence → Test Cases → Scenarios → Requirement

Without traceability, gaps in coverage are invisible until something fails in production.

We understand requirements and traceability.

How does the *testing work itself* proceed? 🤔

STLC — Software Testing Life Cycle

A logical sequence of activities that organizes **how testing is planned, designed, and executed**.

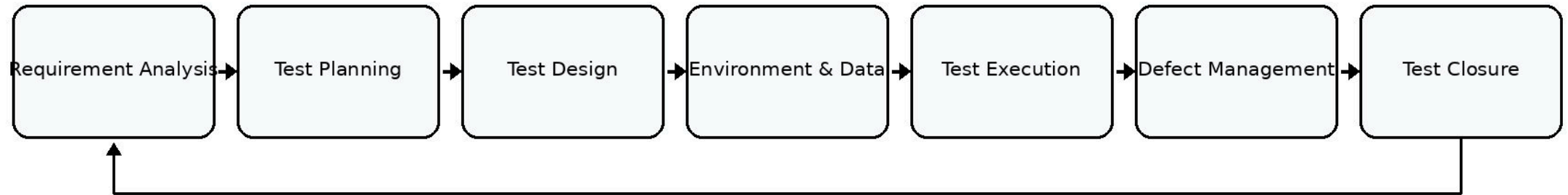
- Provides structure and repeatability
- Ensures testing is traceable, risk-aware, and evidence-driven
- Exact phases and names vary by organization and methodology

 In Agile/DevOps, testing activities happen continuously — but the logical activities of STLC still apply, even if compressed.

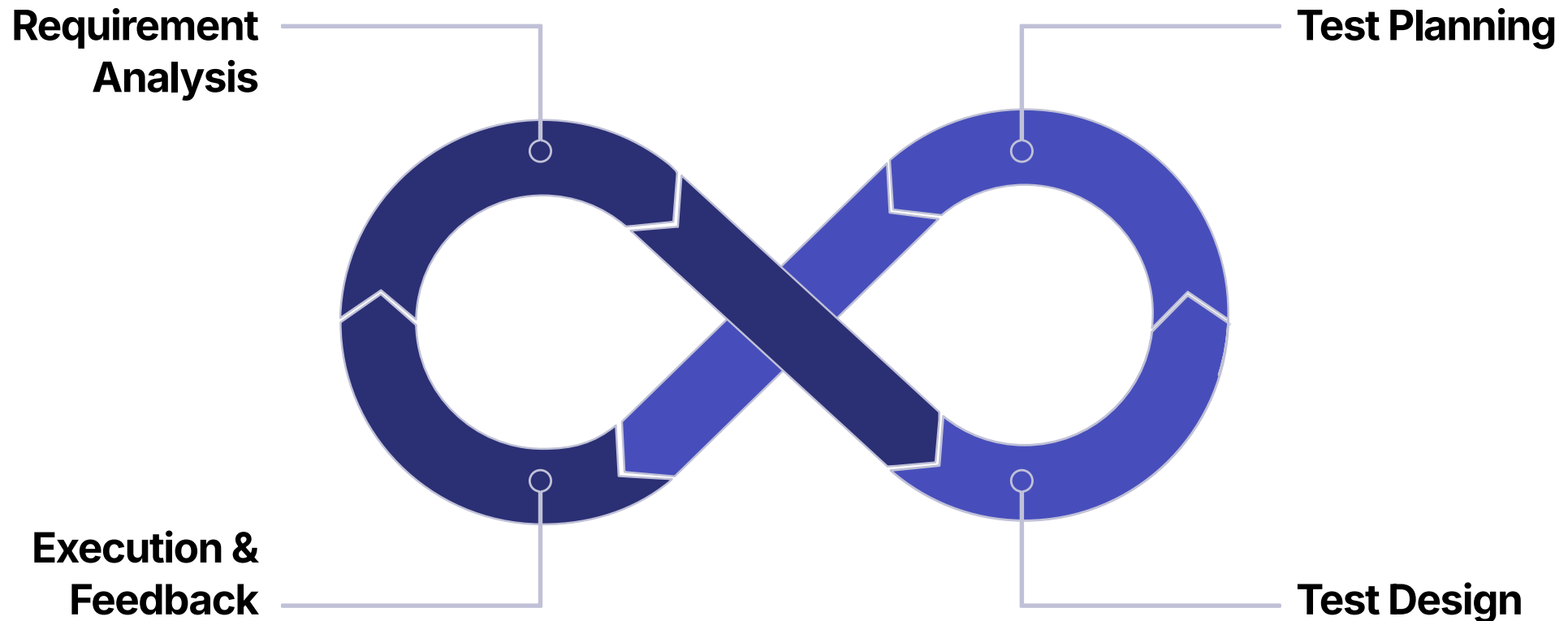
STLC ≠ SDLC

SDLC covers the entire build lifecycle. STLC is the testing thread within it.

STLC — Testing as a Lifecycle



The STLC — Full Lifecycle



Lessons from closure feed back into the next cycle — STLC is iterative, not a one-time waterfall.

Requirement Analysis

1

Understand Scope

What is in scope for this testing cycle?

2

Identify Risks

What areas carry the highest risk if untested?

3

Assess Testability

Can each requirement be verified? What's missing?

4

Map Dependencies

Interfaces, shared data, upstream systems

Output: Shared understanding of what will and won't be tested, and why.

Test Planning



Strategy & Approach

What types of testing? Risk-based? Exploratory?



People & Schedule

Who tests what? When? What are the milestones?



Tools & Resources

Test management, automation, environments needed



Entry & Exit Thinking

What must be true to start? What signals testing is complete?

Output: Test Plan — a living document, not a rigid contract.

Test Design

1 Test Scenarios

WHAT to validate — high-level situations to cover

2 Test Cases / Checks

HOW to validate — conditions, steps, expected results

3 Test Data Needs

What data is required for each scenario?

4 Traceability Links

Map each test case back to its requirement

- ❏ Test Design output varies: formal test case documents in regulated environments; lightweight checklists or exploratory charters in Agile. Both are valid.

Scenario vs. Test Case — know the difference

	Test Scenario	Test Case
Question	WHAT to validate?	HOW exactly to validate?
Detail	High-level situation	Specific conditions, steps, expected result
Example	"Validate transfer limits"	"Transfer ₹49,999 with limit ₹50,000 → expect success; Transfer ₹50,001 → expect E003"
In Agile	Often derived from acceptance criteria	May be exploratory / not formally documented

Test Environment & Test Data

Right Environment

Correct build deployed, services accessible, configuration matches test intent

Right Test Data

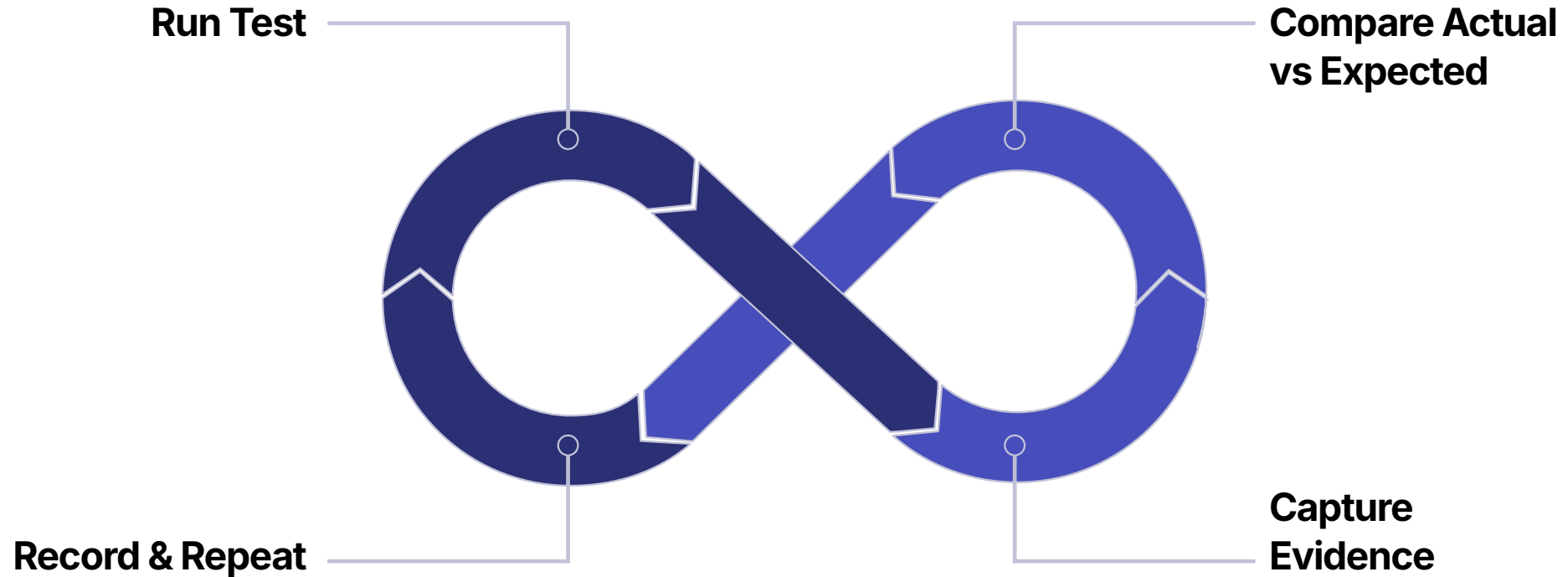
Accounts, states, and values that cover your scenarios — including edge cases

Data Safety

Use synthetic or masked data — never real customer data in lower environments

 The most carefully written test case produces meaningless results if the environment is wrong or the data doesn't represent the scenario.

Test Execution



Evidence matters

Screenshots, logs, API responses — proof that the test ran and what happened

Record everything

Build version, environment, date, tester, actual result

Expected $\neq$ Actual $\rightarrow$ Defect

What QA does

Do NOT just tell a developer verbally. Raise a formal defect with enough information for anyone to reproduce and fix it.

Why formal defect reporting?

Creates a record. Enables tracking, prioritization, root cause analysis, and trend visibility over time.

Good Defect Report — what to include

1

Concise Summary

One clear sentence: what failed and where

2

Environment & Build

Which environment? Which build/version? Browser/OS if relevant

3

Steps to Reproduce

Exact conditions and actions to trigger the defect

4

Expected vs Actual

What should have happened? What actually happened?

5

Evidence

Screenshots, logs, response payloads

6

Severity / Priority Context

Initial assessment — not a final decree; team triages together

Severity vs. Priority — two different questions

Severity

Impact on the system — how badly does this defect break functionality?

Critical / Major / Minor / Trivial

Example: System crashes on every transfer → Critical severity

Priority

Urgency of fix — how quickly must this be addressed?

High / Medium / Low

Example: CEO demo tomorrow → High priority even for a minor defect

 Scales and definitions vary by organization. A minor defect can be high priority. A severe defect can wait. Context matters.

Defect Lifecycle — simplified



Reopen ≠ failure

If a fix doesn't fully resolve the defect, it cycles back. That is the process working correctly.

Workflow varies

Jira, Azure DevOps, ServiceNow — tools differ, but the logical states remain similar.

Test Closure — what QA delivers at the end



Coverage Summary

Requirements tested vs. total; pass/fail rates



Unresolved Risks

Known defects not fixed; areas not tested; assumptions made



Lessons Learned

What can we improve in the next cycle?



Quality Information

Data to inform release decisions — QA advises, business decides

 QA does not "certify zero defects." QA provides quality information. Passing tests do not prove the absence of all defects.

Tests passed on my machine.

**Will the same software
behave identically
everywhere? 🤔**

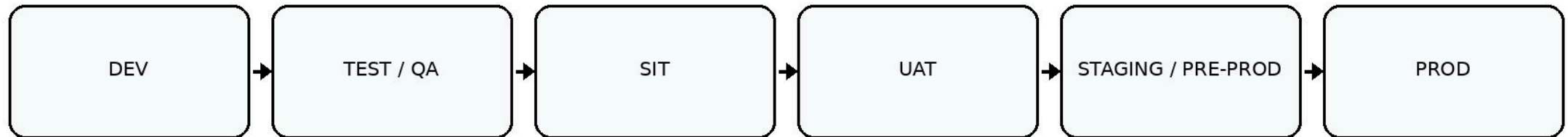
Software Environments — the promotion path



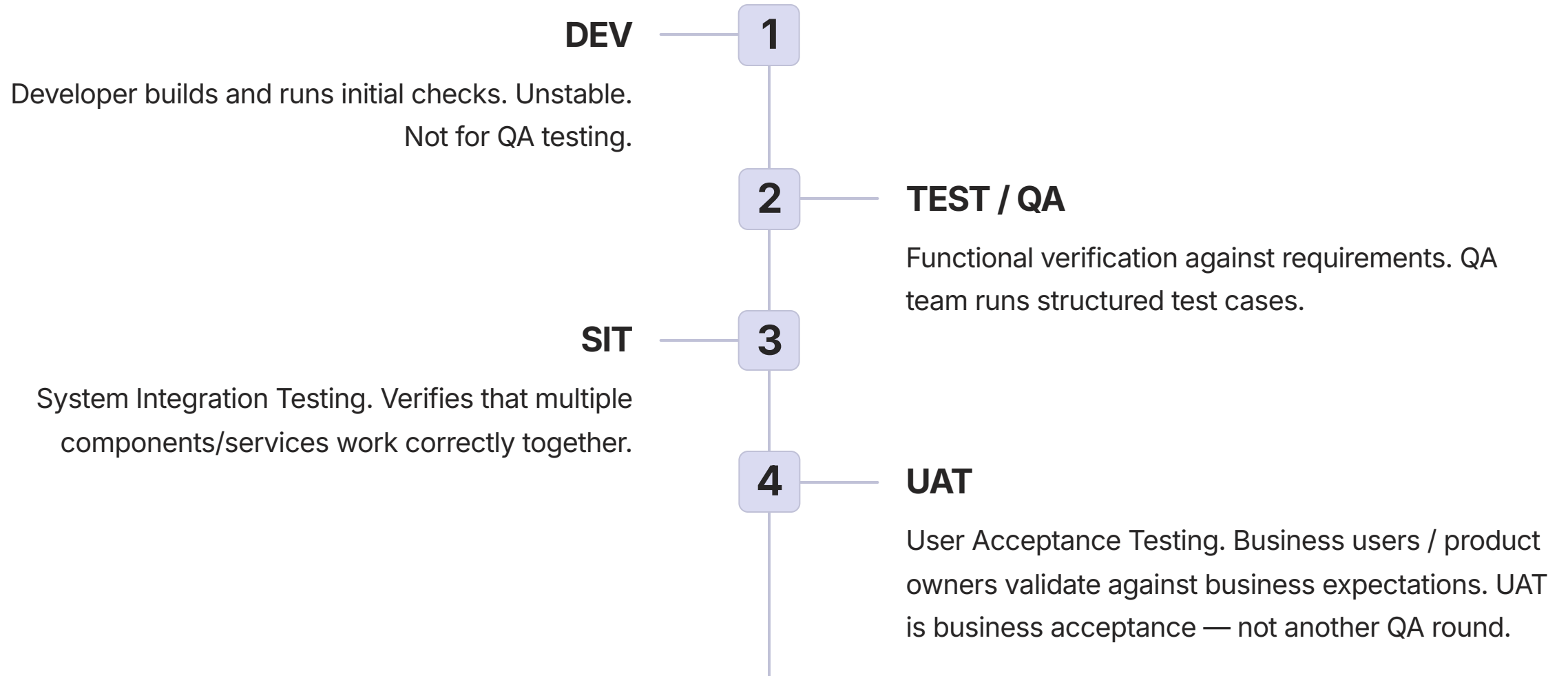
Software doesn't jump from a developer's laptop to production. It earns its way through a series of environments — each with a distinct purpose and audience.

 The number of environments and their names vary by organization. This is a common pattern, not a universal standard.

Environment Promotion — Same Software, Different Purpose



DEV → TEST/QA → SIT → UAT



Staging vs. Production — the final gap

Staging / Pre-Prod

Production-*like* — same stack, similar config, synthetic/masked data

Final gate before release. Smoke tests, performance checks, release validation.

Production

Real users. Real business data. Real consequences.

Configuration, external integrations, data volume, and traffic patterns all differ from any lower environment.



"Works in test, fails in prod" — usually caused by config differences, external dependency behaviour, or data characteristics that lower environments don't replicate exactly.

What kind of system are you testing?

Web UI

Browser rendering, responsive behaviour, form validation, user journeys

Mobile

OS versions, device types, offline behaviour, permissions

API / Backend

Contracts, request/response validation, error codes, performance

Desktop

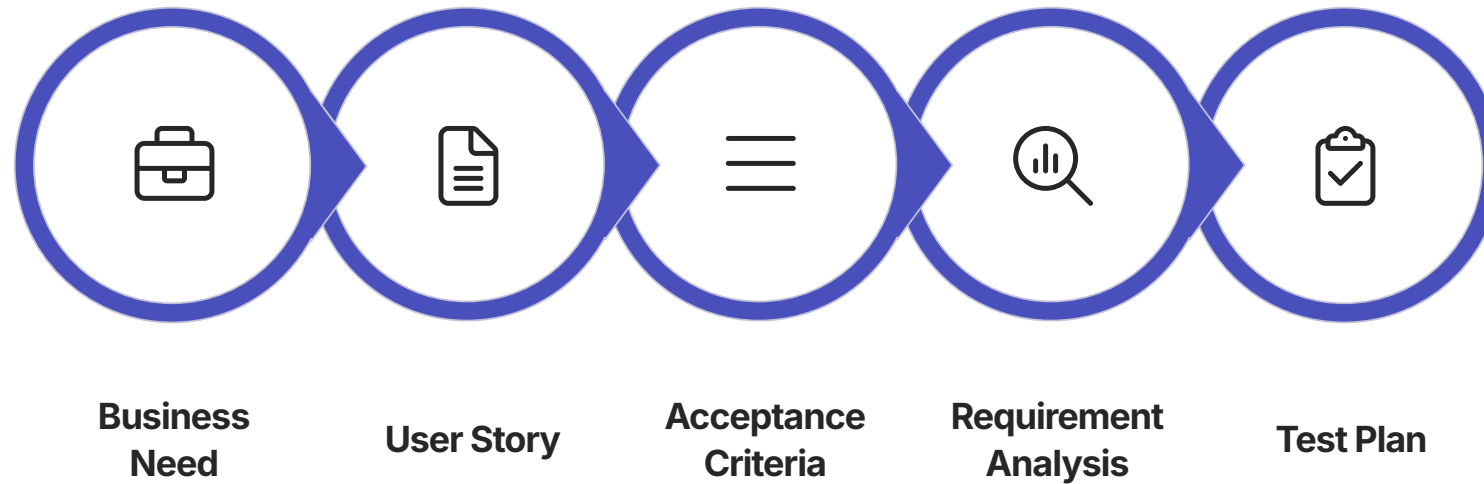
OS compatibility, installation, local data handling

Integrated Systems

End-to-end flows across multiple services, queues, and external dependencies

Each application type exposes different risks. Understanding your system's layers — UI → API → Services → DB → External — guides where to test and what to look for.

From Business Need to Production Confidence



Fresher Challenge

Pick one requirement from the MTS. Can you tell me:

WHAT you would test → **WHERE** you would test it → **HOW** you prove it was tested?

☐ If you can answer all three, you are thinking like a sustain engineer — not just a test executor.